ชีทเปิดสอบ 1 / แผนที่ API

RESTful API · Express 5 · ESM · MySQL

ใช้กับ spec 67130500001–67130500004 · เฉลยร่วมชุดเดียว พร้อมคำอธิบายและแนวฝึกดัดแปลง อ่านหน้า 3 ก่อนเลือกข้อมูลนักศึกษา

> **ลำดับคิด:** Request → Route → Service → Repository → MySQL → DTO → JSON
> ให้ยึด spec และ schema ของตนเองก่อนชีทนี้เสมอ

Method + path ต่อท้าย /api/รหัส	สิ่งที่ต้องทำ	สถานะ
GET /student-info	คืน studentId, name, secretKey	200
GET /brands	Array ของแบรนด์ทั้งหมด	200
GET /brands/:id	แบรนด์ 1 รายการ / ไม่พบ	200 / 404
GET /products?page=1&limit=10	Summary DTO 7 ฟิลด์ + pagination	200
GET /products/:id	Detail DTO + ชื่อแบรนด์	200 / 404
POST /products	เพิ่มสินค้า / SKU ซ้ำ	201 / 409
PUT /products/:id	แก้ไขสินค้า / ไม่พบ ID	200 / 404
DELETE /products/:id	ลบสำเร็จ ไม่มี body / ไม่พบ	204 / 404

ข้อมูลเข้ามาจากไหน?

```
req.params.id     // /products/7 -> '7'
req.query.page    // /products?page=2 -> '2'
req.body.price    // JSON body: {"price": 29900}
req.pagination    // value created by our middleware
```

Pagination ที่ต้องเปิดเจอทันที

```
offset = (page - 1) * limit
totalPages = Math.ceil(total / limit)
hasNextPage = page * limit < total
hasPrevPage = page > 1
```

ตัวอย่าง: total = 75, page = 3, limit = 10 → offset = 20, totalPages = 8, hasNextPage = true, hasPrevPage = true. ตัวเลข 75 เป็นตัวอย่างใน spec ไม่ใช่ค่าที่ต้อง hardcode

เปิดอ่านส่วนไหนเมื่อทำไม่ออก

ส่วน	หน้า
ชีทเร็ว	1–2
เทียบ spec / ขอบเขต	3–4
Setup	5
เฉลย A–H	6–15
ทดสอบ / แก้ Error / ส่งงาน	16–19
แนวโจทย์ดัดแปลง	20–26
โจทย์ฝึก / คำถาม / checklist / แหล่งอ้างอิง	27–30

[S1–S4] §2.1–2.3; สูตรและการจัดลำดับอ่านเป็นคำอธิบายในชีท

ชีทเปิดสอบ 2 / โค้ดจำเป็นและจุดพลาด

คำสั่งเริ่มงาน

```
npm install
npm start
node tests/test-runner.mjs YOUR_STUDENT_ID
```

คำสั่งสุดท้ายใช้ได้เมื่อได้รับชุด tests จริงจากผู้สอนแล้ว เปิดอีก Terminal ขณะที่ Server ยังทำงานที่ port 3000

SQL 5 รูปแบบหลัก

```
select * from brands order by id;
select * from products where id = ?;
select * from products order by id limit ? offset ?;
insert into products (sku, name) values (?, ?);
update products set price = ? where id = ?;
delete from products where id = ?;
```

INSERT ข้างบนเป็นแม่แบบสั้น ๆ ต้องเติมคอลัมน์บังคับตาม schema จริง ค่าใน array ต้องตรงลำดับเครื่องหมาย ? เสมอ

รูปแบบผลลัพธ์ของ mysql2

```
const [rows] = await db.query(sql, values);
const one = rows[0];             // select 1 row
const [result] = await db.query(insertSql, values);
const newId = result.insertId;   // auto-increment key
const count = result.affectedRows;
```

เช็กก่อนเสียคะแนน

อย่าทำแบบนี้	ให้ทำแบบนี้
ส่งทุกคอลัมน์ใน GET /products	แปลงให้เหลือ Summary DTO 7 ฟิลด์
ใช้จำนวนแถวของหน้าปัจจุบันเป็น total	ใช้ select count(*) นับข้อมูลทั้งหมดที่เข้าเงื่อนไข
ลบสำเร็จแล้วส่ง JSON	res.status(204).end()
if (!data.stock_quantity)	ตรวจ undefined/null; ค่า 0 เป็นข้อมูลที่มีค่า
ใช้ changedRows = 0 แปลว่าไม่พบ ID	ตรวจว่ามีแถวนั้นก่อน update; ค่าเดิมก็สำเร็จได้
คัดลอกชื่อ/Secret Key ของเพื่อน	ใช้ข้อมูลใน spec ที่ตนเองได้รับ
เช็ก SKU ก่อน INSERT อย่างเดียว	ยังต้องมี UNIQUE ที่ฐานข้อมูลเพื่อกันคำขอพร้อมกัน

> **แยก 400 / 404 / 409:** ข้อมูลคำขอไม่ถูกต้อง / ไม่พบ resource / ข้อมูลขัดแย้ง เช่น SKU ซ้ำ ส่วน 500 คือปัญหาฝั่ง Server ที่ต้องอ่าน log

[S1–S4] §2–4; [R2] mysql2 Promise; [R7] HTTP status